Vuln-Bank API Penetration Testing Report

Prepared by: *Annie Stephen Bassey*
Date: *17th Decmber 2025*
Target Application: vuln-bank API
Test Environment: Local postgres (Kali Linux)
Testing Type: API Security Assessment

---

1. Executive Summary

This report documents a security assessment conducted against the **vuln-bank API** . The objective of the test was to identify weaknesses aligned with the **OWASP API Security Top 10**, with a specific focus on input handling, configuration issues, and trust boundaries.

Testing was performed using **Postman** from a Kali Linux environment and a simulated attacker interacting directly with the backend API. Multiple security issues were identified that could allow attackers to manipulate application behavior, leak sensitive information, or abuse business logic.

Key Findings Summary

Vulnerability	Severity	OWASP API Mapping
Mass Assignment	High	API6
CORS Misconfiguration	Medium	API7
Injection (Error-Based)	Medium	API8
Unsafe Consumption of APIs	Medium	API10

If exploited, these issues could result in unauthorized balance manipulation, information disclosure, and abuse of core banking functionality.

---

2. Scope

In Scope

- vuln-bank API (http://localhost:5000)
- Authentication and account-related endpoints
- User registration and login functionality

---

3. Methodology

The assessment followed a **manual API penetration testing approach**, aligned with OWASP API Security guidelines.

Tools Used

- Postman (manual request crafting and testing)
- postgres(local application deployment)
- Kali Linux (testing environment)

Testing Approach

1. Identified available API endpoints
2. Established baseline authenticated and unauthenticated behavior
3. Manipulated request payloads and headers
4. Observed responses, error messages, and headers
5. Collected request/response evidence for each finding

4. Key Findings

Finding 1: Mass Assignment

OWASP Category: API6 – Mass Assignment
Severity: High

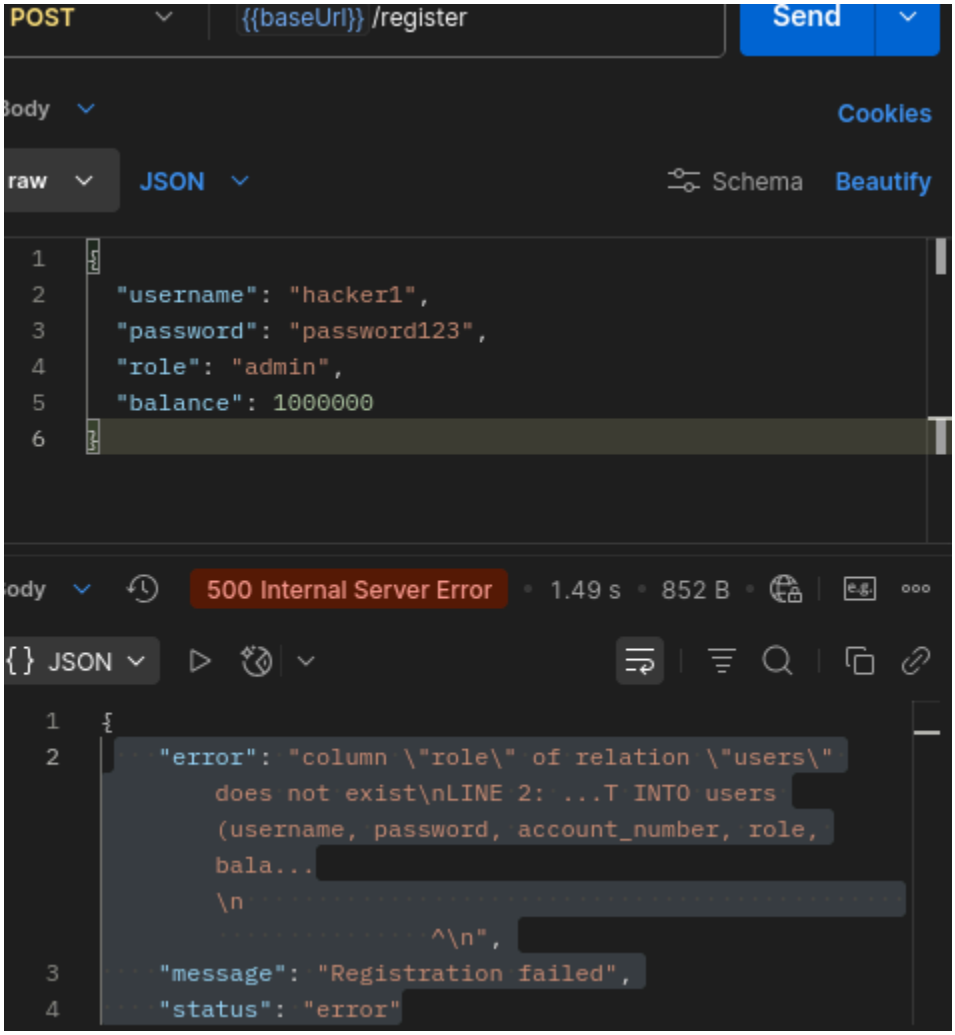
Description

The user registration endpoint accepts user-supplied JSON data and attempts to process all provided fields without proper allow-listing. When unexpected fields were supplied, the backend attempted to insert them directly into the database.

Affected Endpoint

`POST /api/register`

Proof of Concept



Observed Behavior

The API attempted to insert the `role` field into the database, resulting in a SQL error disclosing internal schema details.

Security Impact

If sensitive fields (such as balance or privilege flags) exist in the database, attackers could manipulate them during account creation, leading to privilege escalation or financial abuse.

Finding 2: CORS Misconfiguration

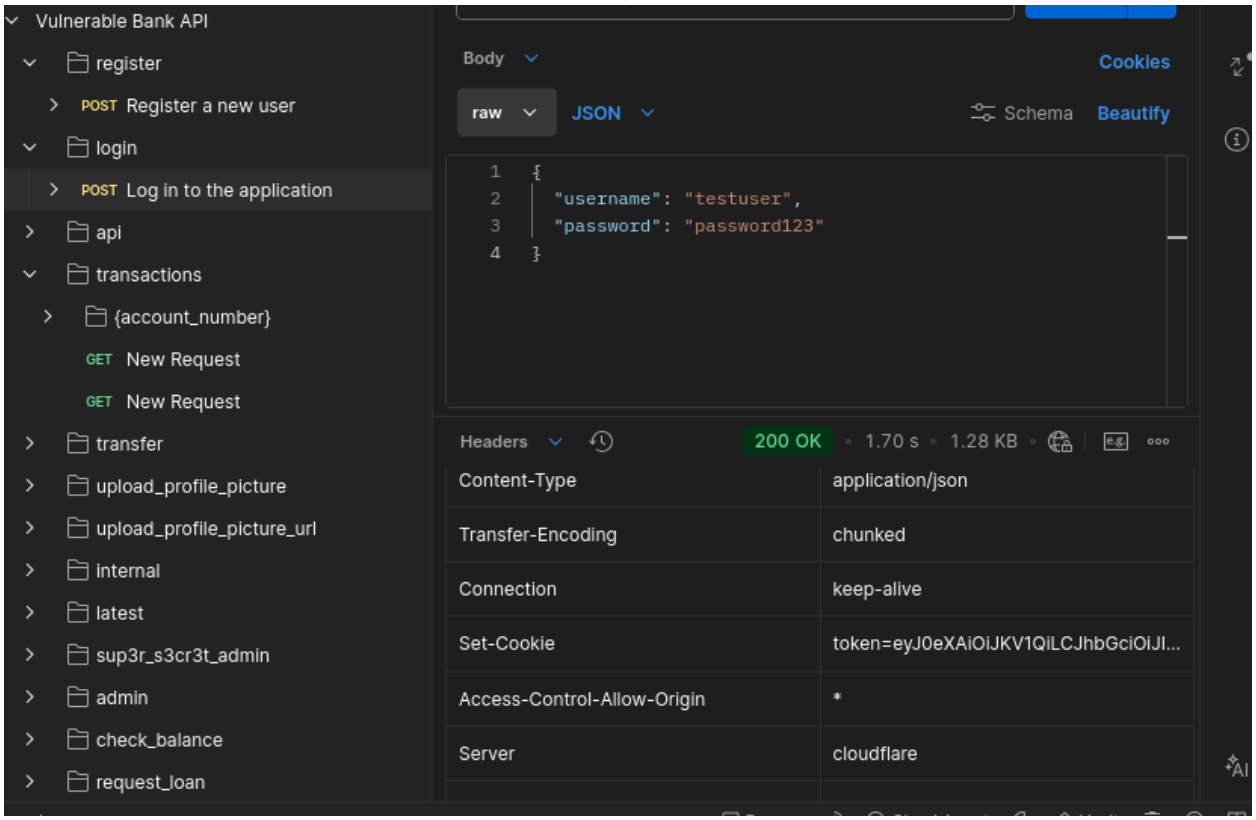
OWASP Category: API7 – Security Misconfiguration
Severity: Medium

Description

The API responds with permissive CORS headers, allowing cross-origin requests from untrusted origins.

Evidence

Response headers included:



Security Impact

A malicious website could issue authenticated requests on behalf of a logged-in user, potentially allowing data theft or unauthorized actions via cross-site attacks.

Finding 3: Injection (Error-Based)

OWASP Category: API8 – Injection
Severity: Medium

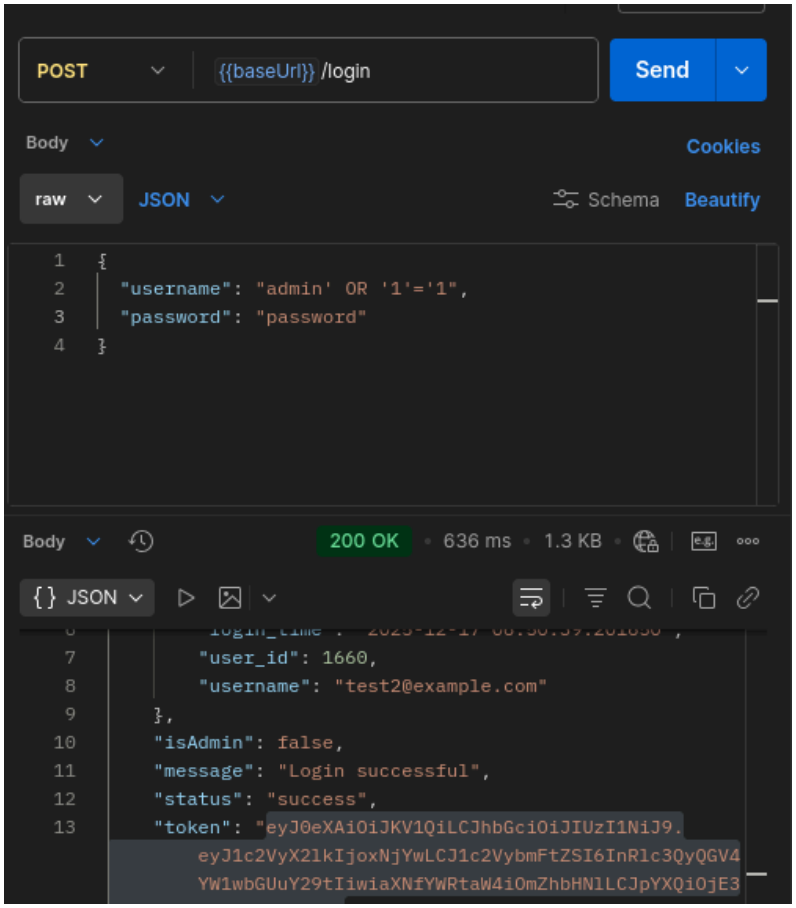
Description

The application returns a status of 200, implying that the input supplied was validated,but logged in as a different user. This behavior indicates insufficient input validation and improper error handling.

Affected Endpoint

`POST /api/login`

Evidence



Security Impact

Detailed error messages provide attackers with insight into the database structure, which can be leveraged to craft more advanced attacks such as SQL injection or mass assignment exploitation.

Finding 4: Unsafe Consumption of APIs

OWASP Category: API10 – Unsafe Consumption of APIs
Severity: Medium

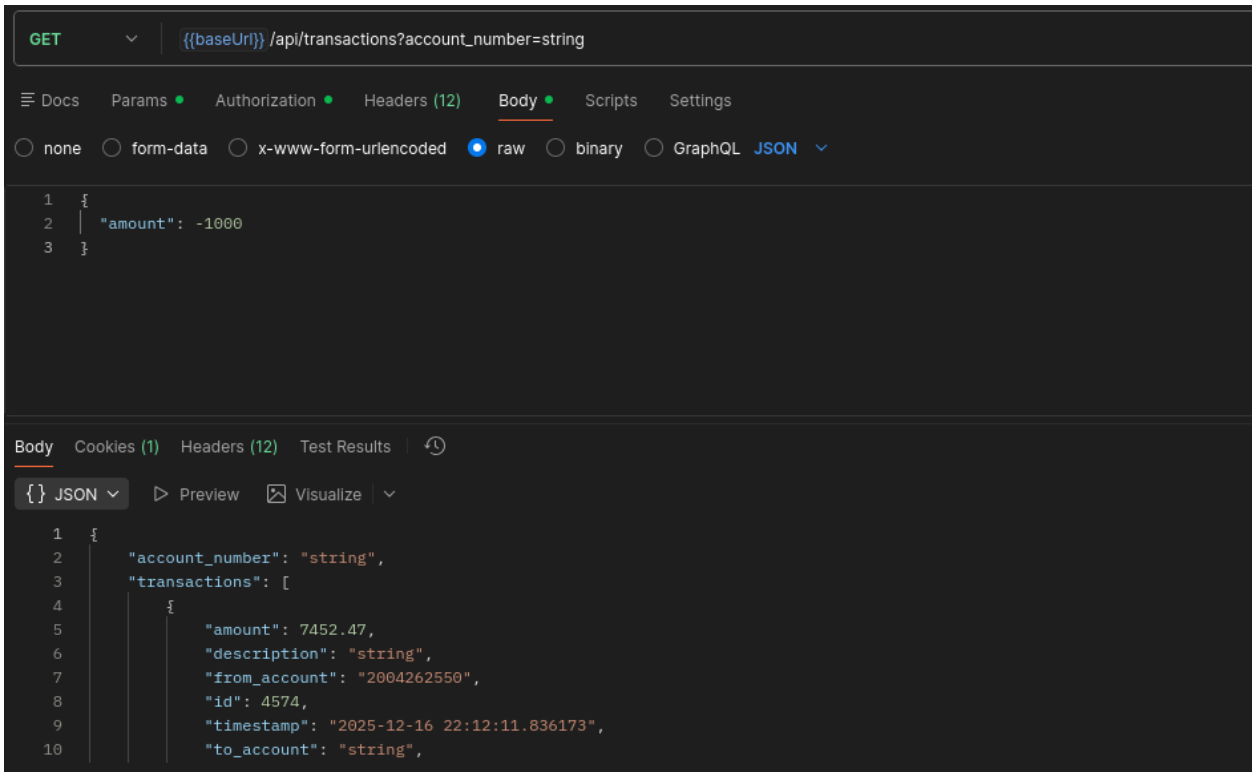
Description

The application accepts and processes raw transaction data from an external API without proper validation or sanitization. This allows manipulation of transaction amounts and other sensitive fields, leading to potential business logic bypass or financial fraud.

Affected Endpoint:

GET /api/transactions?account_number=string

Proof of Concept



Supplying unexpected values such as negative or excessively large transaction amounts was accepted by the backend.

Security Impact

Attackers can abuse business logic to manipulate account balances, perform invalid transactions, or bypass intended financial constraints.

---

5. Recommendations

Vulnerability	Recommendation
Mass Assignment	Implement strict server-side allow-listing for accepted fields. Reject unknown properties.

CORS Misconfiguration	Restrict `Access-Control-Allow-Origin` to trusted domains only.
Injection	Disable verbose error messages in production. Implement centralized input validation and error handling.
Unsafe API Consumption	Enforce server-side validation for all business logic values (e.g., transaction limits, non-negative balances).

---

6. Conclusion

The vuln-bank API demonstrates common security weaknesses frequently observed in real-world APIs, particularly around input trust and configuration management. While authentication mechanisms are present, insufficient validation and overly permissive configurations expose the application to multiple attack vectors.